

 poolside/**Laguna-S-2.1-NVFP4** 

 like 114

Follow  Poolside 955

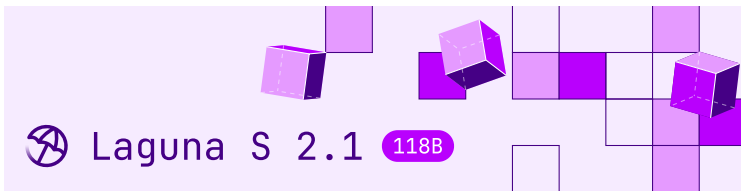
 Text Generation
  Safetensors
  vllm
  laguna
  laguna-s-2.1
  conversational
  custom_code

 8-bit precision
  compressed-tensors
  License: openmdw-1.1

⋮

→ Copy to bucket **NEW**

 **Model card**
 Files
  xet
  Community 13



[Use on OpenRouter](#) · [Use on Vercel AI Gateway](#) ·
[Release blog post](#)

Laguna S 2.1-NVFP4

Laguna S 2.1-NVFP4 is a 117.6B total parameter Mixture-of-Experts model with 8.5B activated parameters per token designed for agentic coding and long-horizon work on a local machine. It uses Sliding Window Attention with per-head gating in 36 out of 48 layers for fast inference and low KV cache requirements.

Highlights

- Mixed SWA and global attention layout:**
 Laguna S 2.1 uses softplus gating with per-layer rotary scales, enabling mixed SWA (Sliding

Downloads last month
52,235



Safetensors

Model size 68B params

Tensor type F32 · BF16 · F8_E4M3 · U8

 Chat template
  Files info

Inference Providers **NEW**

 Text Generation

This model isn't deployed by any Inference Provider.

 Ask for provider support

Model tree for poolside/Laguna-S...

Base model poolside/Laguna-S-2.1

 Quantized (56) [this model](#)

Finetunes [1 model](#)

Quantizations    [1 model](#)

 Collection including poolside/Lag...

Window Attention) and global attention layers in a 3:1 ratio (across 48 total layers)

- **KV cache in FP8:** KV cache quantized to FP8, reducing memory per token
- **Native reasoning support:** Interleaved thinking between tool calls with support for enabling and disabling thinking per-request
- **Local-ready:** At 117.6B total parameters and 8.5B activated, the NVFP4 weights are roughly 71 GB. Available on [Ollama](#) and [llama.cpp](#) (BF16 and Q4_K_M only)
- **OpenMDW-1.1 license:** Use and modify the model and associated materials freely for commercial and non-commercial purposes ([learn more about OpenMDW](#))

Model overview

- Training: pre-training, post-training and reinforcement learning stages
- Number of parameters: 117.6B total with 8.5B activated per token
- Optimizer: Muon
- Layers: 48 layers (12 layers with global attention, 36 layers with sliding window attention)
- Experts: 256 experts with 1 shared expert
- Sliding Window: 512 tokens
- Modality: text-to-text
- Context window: 262,144 tokens

- Reasoning support: interleaved thinking with preserved thinking

Benchmark results



Model	Size	Terminal-Bench 2.1	SWE-bench Multilingual	SWE Bench Pro (Public Dataset)
Laguna S 2.1	118B-A8B	70.2%	78.5%	59.4%

Model	Size	Terminal-Bench 2.1	SWE-bench Multilingual	SWE Bench Pro (Public Dataset)
Tencent Hy3	295B-A21B	71.7%	75.8%	57.9%
Inkling	975B-A41B	63.8%	-	54.3%
Nemotron 3 Ultra	550B-A55B	56.4%	67.7%	-
DeepSeek-V4-Pro Max	1.6T-A49B	64.0%*	76.2%	55.4%
Kimi K3	2800B-A50B	88.3%	-	-
Qwen 3.7 Max	-	74.5%*	78.3%	60.6%
Muse Spark 1.1	-	80%	-	61.5%
Claude Fable 5	-	88%	-	80.3%

Benchmarks as of 21 July 2026. Laguna S 2.1 in **bold**; a dash (-) marks a benchmark a model was not evaluated on. Scores marked * are as reported by third parties: Terminal-Bench 2.1 and DeepSWE via Artificial Analysis, SWE Atlas via Scale AI's official leaderboard, and Toolathlon Verified via its official leaderboard. Full evaluation trajectories: trajectories.poolside.ai.

Context length

This checkpoint ships configured for a 262,144-token (256K) context window. This is the configuration we recommend for best output quality.

The weights are native 1M checkpoints: training included a long-context extension stage up to 1,048,576 tokens, and quantization was calibrated at the 1M configuration. If you need more than 256K of context, restore the 1M configuration by editing `config.json`:

```
"rope_parameters": {  
  "full_attention": {  
    "factor": 128.0,  
    "attention_factor": 1.4852030263919618  
  }  
},  
"max_position_embeddings": 1048576
```

You may experience quality degradation with the 1M configuration. If you use it, we recommend sampling with `temperature <= 0.7` and `top_p <= 0.95`.

Recommended sampling

For the best balance of quality and reliability we recommend sampling with `temperature 0.7` and `top_p 0.95`.

Local deployment

Laguna S 2.1-NVFP4 is supported in vLLM, SGLang and Transformers, and TRT-LLM thanks to the

support of the team at NVIDIA. Use Laguna-S 2.1 with Ollama (with MLX support) or Llama.cpp (BF16 and Q4_K_M only) for the best results on your local machine.

vLLM

“Serving requires vLLM 0.25.0 or later. See the main [Laguna S 2.1 model card](#) for the full recipe.”

The full vLLM recipe is on the main [Laguna S 2.1 model card](#) and on the [vLLM recipes page](#).

Quantization is detected automatically from `quantization_config` in this checkpoint, so the same command works with `poolside/Laguna-S-2.1-NVFP4` substituted for the model ID. No extra flags required.

Optional: speculative decoding with DFlash. Pair with the quantization-matched draft model [poolside/Laguna-S-2.1-DFlash-NVFP4](#) by adding `--speculative-config '{\"model\": \"poolside/Laguna-S-2.1-DFlash-NVFP4\", \"num_speculative_tokens\": 15, \"method\": \"dflash\"}'` to the [serve command](#).

DGX Spark & Mac Studio

Ollama is the quickest way to run Laguna S 2.1 on a single 128 GB box, whether that is an NVIDIA DGX Spark (GB10) or an Apple Silicon Mac Studio:

```
ollama run laguna-s-2.1
```

It pulls the Q4_K_M GGUF (about 75 GB) and runs on the GB10 GPU on the Spark or on Metal on the Mac,

with no build step. We measured about 12.6 tokens/s on the Spark and 17.6 on Apple Silicon.

- You need about 128 GB of unified memory; the Q4_K_M weights are around 75 GB on their own.
- The first load reads 75 GB off disk and can run past Ollama's 5-minute default timeout. If it does, set `OLLAMA_LOAD_TIMEOUT=20m`.
- Reasoning is on by default. Pass `"think": false` in the request to turn it off.
- The higher-fidelity tags need a bigger box: `laguna-s-2.1:q8_0` is about 128 GB (so a 192 GB Mac Studio) and `laguna-s-2.1:f16` is about 235 GB (256 GB or more). On the Spark's 128 GB, stick to Q4_K_M. Apple Silicon can also run it through MLX.
- Ollama is built on llama.cpp. To build the runtime yourself, poolside's [laguna branch of llama.cpp](#) serves the same GGUFs.

Maximum performance on the DGX Spark (native NVFP4 + DFlash)

For the fastest setup on the Spark, serve this checkpoint with vLLM instead. That gives you the native NVFP4 kernels and DFlash speculative decoding.

One-time setup:

```
# Python headers: Triton JIT needs them and ,
# Without this, the first server start dies .
sudo apt install -y python3.12-dev

curl -LsSf https://astral.sh/uv/install.sh |
uv venv ~/venvs/vllm025 -p 3.12
```

```
# vLLM 0.25.1 with CUDA-13 torch (aarch64 wh
uv pip install -p ~/venvs/vllm025 vllm==0.25
```

```
# FlashInfer nightly trio: without flashinfe.
# native; the jit-cache wheel avoids most fi.
uv pip install -p ~/venvs/vllm025 \
    "flashinfer-python==0.6.15.dev20260712" \
    "flashinfer-cubin==0.6.15.dev20260712" \
    "flashinfer-jit-cache==0.6.15.dev20260712"
--extra-index-url https://flashinfer.ai/wh
--extra-index-url https://flashinfer.ai/wh
--index-strategy unsafe-best-match
```

```
hf download poolside/Laguna-S-2.1-NVFP4
hf download poolside/Laguna-S-2.1-DFlash-NVFI
```

Serve:

```
export CUTE_DSL_ARCH=sm_121a          # arch
export PATH=/usr/local/cuda/bin:$PATH # nvcc
export MAX_JOBS=4                      # cap
source ~/venvs/vllm025/bin/activate
```

```
vllm serve poolside/Laguna-S-2.1-NVFP4 \
    --speculative-config '{"model": "poolside/La
    --enable-auto-tool-choice \
    --tool-call-parser poolside_v1 \
    --reasoning-parser poolside_v1 \
    --override-generation-config '{"temperature
    --max-num-seqs 32 \
    --max-model-len 262144 \
    --gpu-memory-utilization 0.85 \
    --host 0.0.0.0 --port 8000
```

- You do not need backend flags: auto-selection picks FlashInferCutlass, which runs natively on sm_121. Do not set --linear-backend

flashinfer_b12x on 0.25.1; the opt-in is broken there and it is slower anyway.

- Keep the `--override-generation-config`. Many clients send no sampling parameters, and the raw defaults degrade output on the NVFP4 quantization. The model's `generation_config.json` sets `top_k 20` (eval-certified truncation), which handles that. Do not add `min_p`: vLLM rejects `min_p` and `logit_bias` under speculative decoding, so putting it in the defaults returns a 400 on every sampled request.
- `--max-num-seqs 32` is required: DFlash crashes vLLM at the default of 256.
- The first start takes about 15 minutes (weight load from NVMe, JIT, and graph capture).

Never drop `MAX_JOBS=4` on a cold `~/.cache/flashinfer`. An uncapped nvcc fan-out can exhaust the 128 GB unified memory and take the whole machine down. A warm cache makes it a no-op, but the cache is cold again after any config or shape change.

Prefill runs 600-800 tok/s, decode is around 15 tok/s on prose and 22-24 on code, and DFlash accepts 2.9-3.1 tokens per step. At the 256K setting you get 830-870K KV tokens. Two requests at once roughly double total throughput. Without speculation, decode sits at 13-14 tok/s on every engine we tried on the GB10, which is the memory-bandwidth ceiling for this model. Speculation is how you get past it.

pool CLI against the box:

```
POOLSIDE_STANDALONE_BASE_URL=http://<spark-ip>:4000
POOLSIDE_STANDALONE_MODEL=poolside/Laguna-S-2.1-NVFP4
POOLSIDE_STANDALONE_CONTEXT_LENGTH=262144 \
POOLSIDE_API_KEY=dummy pool
```

Use the box's IPv4 address rather than the `.local` mDNS name: Go's resolver can pick the link-local IPv6 and fail with "no route to host". On macOS the terminal app needs Local Network permission, and the system curl is exempt from that check, so if curl works but pool does not, it is the permission and not the network.

SGLang

Laguna S 2.1 is supported in SGLang via [sgl-project/sglang#24204](#). Quantization is detected automatically from `quantization_config`, so no extra flags are required. See the [SGLang cookbook entry](#) and the main [Laguna S 2.1 model card](#) for a serving recipe.

Transformers

The full Transformers recipe is on the main [Laguna S 2.1 model card](#). Substitute `poolside/Laguna-S-2.1-NVFP4` for the model ID; quantization is detected automatically from `quantization_config`.

TRT-LLM

Laguna S 2.1 support ships in TensorRT-LLM `>=1.3.0rc16`; see the install recipe on the main [Laguna S 2.1 model card](#). Substitute `poolside/Laguna-S-2.1-NVFP4` for the model ID; quantization is detected automatically from `quantization_config`, no extra flags required.

```
from tensorrt_llm import LLM
```

```
llm = LLM(model="poolside/Laguna-S-2.1-NVFP4
```

Ollama

Available on the [Ollama library](#).

Controlling reasoning

Laguna S 2.1-NVFP4 uses the same reasoning controls (interleaved thinking, preserved reasoning, and the `enable_thinking` flag) as the base model. See the [Controlling reasoning](#) section of the main Laguna S 2.1 model card.

License

This model is licensed under the [OpenMDW-1.1 License](#).

Intended and Responsible Use

Laguna S 2.1-NVFP4 is designed for software engineering and agentic coding use cases, and you are responsible for confirming that it is appropriate for your intended application. Laguna S 2.1-NVFP4 is subject to the [OpenMDW-1.1 License](#), and should be used consistently with Poolside's [Acceptable Use Policy](#). We advise against circumventing Laguna S guardrails without implementing substantially equivalent mitigations appropriate for your use case.

Please report security vulnerabilities or safety concerns to security@poolside.ai.



System theme



TOS



Privacy



About



Careers



Models

Datasets

Spaces

Pricing

Docs